



CEIPFP Ceste

Baldur's Bones

Memoria del Proyecto Intermodular

Ciclo Formativo de Grado Superior
Desarrollo de Aplicaciones Multiplataforma

Curso académico: 2025/2026

Alumno/a:

Hugo Tarín González

Tutor/a:

Borja Aparisi Navarro

Índice

1. Introducción	2
1.1. Justificación	2
1.2. Objetivos	2
2. Tecnologías	3
2.1. Unity	3
2.1.1. NavMesh	3
2.1.2. UINavigation	4
2.1.3. InputSystem	4
2.1.4. Blend Tree	5
2.1.5. PlayerPrefs	5
2.2. Relación con asignaturas	6
2.2.1. Desarrollo de Interfaces	6
2.2.2. Acceso a Datos	6
2.2.3. Programación Multimedia y Dispositivos Móviles	6
2.2.4. Programación de Servicios y Procesos	6
3. Desarrollo del proyecto	8
3.1. Ambientación	8
3.2. Sonido	9
3.2.1. MusicManager.cs	9
3.2.2. SoundManager.cs	9
3.3. Escenas	10
3.3.1. Flujo entre escenas	10
3.4. MenuSystem.cs	11
3.5. Escena “Menu”	12
3.5.1. Diseño	12
3.5.2. Implementación	12
3.6. Escena “Mercado”	13
3.6.1. CoinsSystem.cs	13
3.6.2. UIManager.cs	13
3.6.3. Diseño	13
3.6.4. Implementación (MarketSystem.cs)	14
3.7. Escena “Juego”	15
3.7.1. Diseño	15
3.7.2. Implementación	15
3.7.3. DayNightCycle.cs	16
3.8. Escena “GameOver”	17
3.8.1. Diseño	17
4. Bugs y soluciones	18
4.1. SceneAdditive	18

1. Introducción

Este proyecto consiste en un videojuego multiplataforma y multimedia, desarrollado en Unity, que contiene sistemas de movimiento basado en vectores, de gestión de escenas y de sonidos, además de scripts dedicados a detalles para que sea lo más cómodo y visual para el usuario.

1.1. Justificación

Al comienzo del curso iba a desarrollar junto con otro alumno una plataforma de gestión de flujos de trabajo, pero con el paso de las semanas decidimos cambiar y hacer proyectos por separado, yo me decanté por llevar más allá el videojuego de PMDM, ya que ya había tenido la ilusión de programar un videojuego, y decidí que era el momento idóneo.

1.2. Objetivos

Los objetivos principales del proyecto son los siguientes:

- Desarrollar una UI/UX satisfactoria para el usuario.
- Implementar un sistema de guardado de progreso para los datos del videojuego.
- Crear un flujo de juego equilibrado y con avances en el tiempo.
- Generar una ambientación y entorno inmersivo.
- Tener la opción de jugar en PC, móvil y con mando.

2. Tecnologías

2.1. Unity

La principal herramienta de desarrollo es Unity, ya que facilita infinitamente la tarea de desarrollar un videojuego, con muchas facilidades que incorpora que ha hecho viable el desarrollo y el cumplimiento de este proyecto.

Algunas de estas tecnologías que ofrece Unity y he implementado en el proyecto son las siguientes:

2.1.1. NavMesh

Es un sistema basado en IA que genera una malla sobre un objeto con una capa específica que utilizaremos como terreno (NavMesh), para permitir que nuestros enemigos (NavMeshAgent), lleguen a nuestro personaje (`agent.SetDestination(personaje)`), usando algoritmos de IA para encontrar el camino más rápido evitando obstáculos (NavMeshObstacle) o simplemente objetos demasiado grandes como para superarlos (configuración de los Agents). Además de crear links (OffMeshLink) para ir de un lado del NavMesh a otro atajando.

En esta imagen se puede observar perfectamente la malla que crea NavMesh y como afectan los demás elementos del sistema.

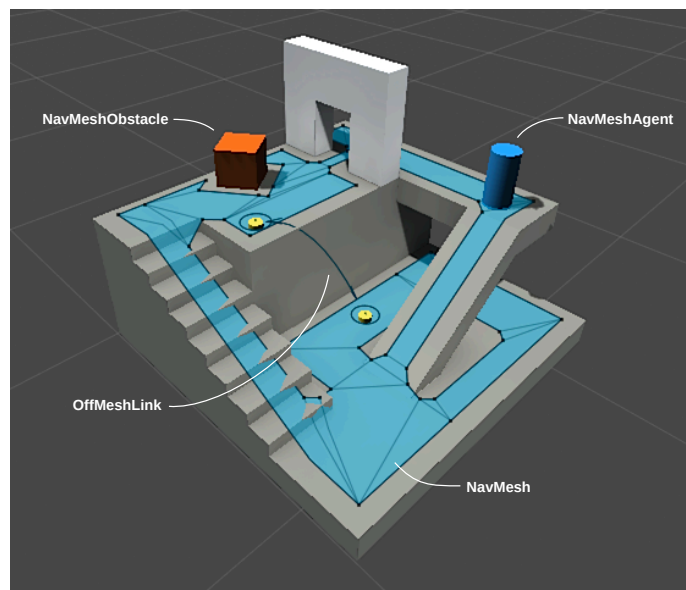


Figura 1: Imagen visual de los elementos del NavMesh

Esto ha facilitado el apartado de la jugabilidad, que afecta directamente al objetivo de una UX satisfactoria.

2.1.2. UINavigation

Es el sistema que controla la selección de elementos en la pantalla que pueden ser accionados (Interactable) en la escena, teniendo la opción de establecer los cambios a elementos vecinos de manera automática o manual.

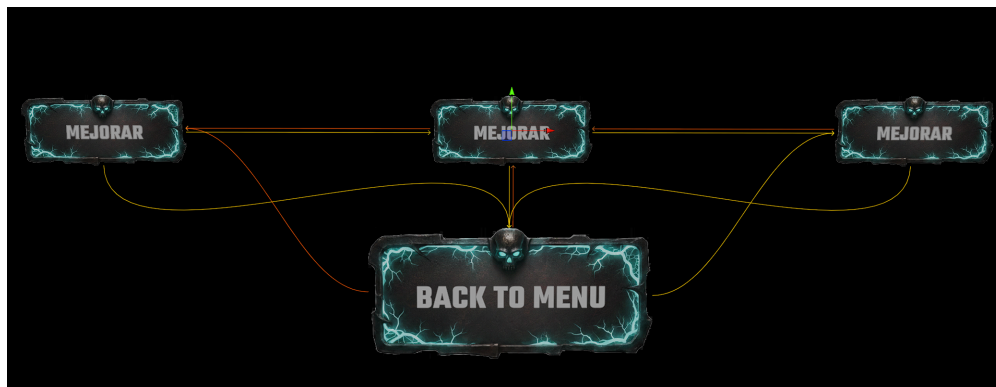


Figura 2: Imagen real del flujo entre botones generados por UI Navigation del proyecto

Esta función me hizo extremadamente fácil añadir la funcionalidad de los menús con el mando.

2.1.3. InputSystem

Es el último sistema para detectar entradas de cualquier dispositivo de Unity. Funciona con un concepto de acciones, las cuales se usan para realizar acciones en el código.

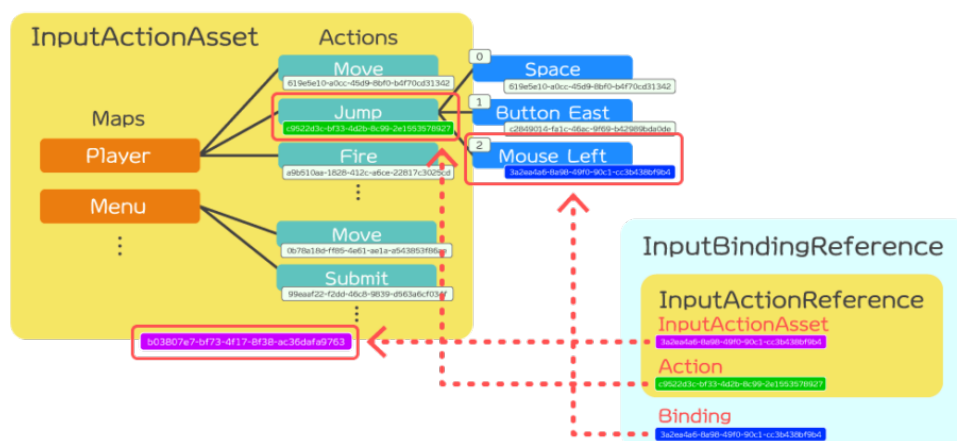


Figura 3: Imagen explicativa del uso de InputSystem

En esta imagen se explican los conceptos de Maps, que agrupan una serie de Actions, las cuales tienen asociadas unos Bindings, que son las entradas de diferentes dispositivos que activan esa Action.

Se puede llamar al valor que devuelve cada acción por la siguiente estructura:

```
InputSystem_Actions.<Map>.<Action>.<Method>
```

Pudiendo ser Method un `isPressed()` o `ReadValue()`, según el tipo de dato que devuelva esa acción.

Esta tecnología me permitió implementar la funcionalidad de multiplataforma de una manera mucho más sencilla, ya que para implementar el ataque de mi personaje con el mando, por

ejemplo, simplemente tenía que agregar el Binding correspondiente del botón que quiero que active la Action de Ataque.

2.1.4. Blend Tree

Es una tecnología que vive dentro del Animator, que es el sistema que gestiona las animaciones de los objetos. Específicamente Blend Tree se encarga de mezclar animaciones según unos parámetros. Si solo se utiliza un parámetro es un Blend Tree 1D, si utiliza 2 es un Blend Tree 2D.

Un ejemplo de esto puede ser el cambio de animación entre una animación Caminar y Correr, basándose en parámetro Velocidad. Por defecto es proporcional el aumento del uso de una animación u otra, en 0.5 de Velocidad utilizará el 50% de los datos de cada animación, pero esto puede cambiarse.

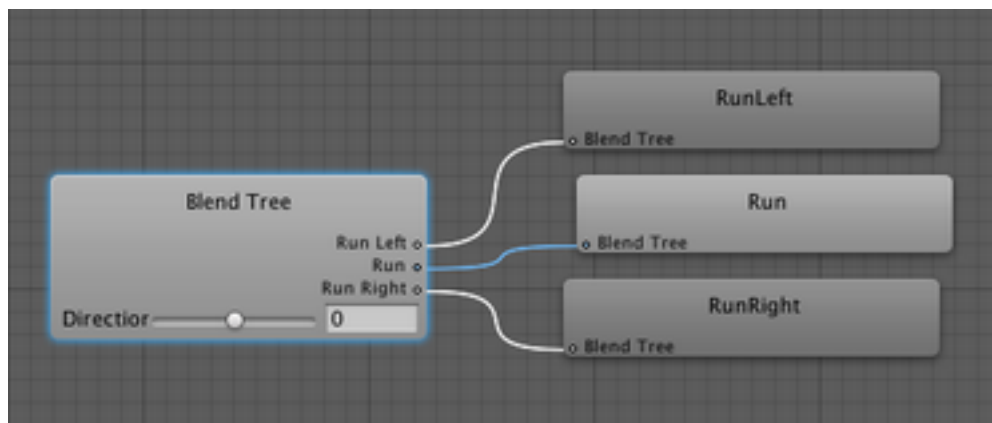


Figura 4: Imagen sobre otro ejemplo de uso de Blend Tree 1D

Esta tecnología me ayudó a crear una UX mucho más amigable y visual para el usuario, que de otra manera habría sido imposible, sabiendo mi poca experiencia en este sector.

2.1.5. PlayerPrefs

Este es el sistema más simple que ofrece Unity para guardar y cargar datos de forma persistente en la memoria del dispositivo. Tiene un formato clave-valor, aunque solo cuenta con tres tipos de datos, Int, Float y String.

Así se guarda un valor en la memoria del dispositivo:

```
PlayerPrefs.SetInt("monedas", 150);  
// Asina a la clave "monedas" el valor de 150 y la guarda en el sistema
```

Y así se carga un valor de la memoria al código:

```
int monedas = PlayerPrefs.GetInt("monedas", 0); // Devuelve 0 por defecto  
// Busca el valor de la clave "monedas" y si no la encuentra devuelve el  
valor por defecto
```

Esta funcionalidad fue vital para poder hacer de manera simple y eficaz uno de los grandes objetivos del proyecto, un sistema de guardado de datos para mantener el progreso del usuario.

2.2. Relación con asignaturas

2.2.1. Desarrollo de Interfaces

Las interfaces están en cada parte del juego, sinceramente les he puesto esmero, sobretodo en el apartado de multiplataforma, quería que la versión para ordenadores fuese jugable y cómoda, sin elementos que estorben, a la vez de que la de móviles fuese simple e intuitiva, sin dificultades.

Por lo que decidí hacer una versión única para cada dispositivo únicamente por esta funcionalidad, y mejorar la UX.

En cada una de las escenas del videojuego hay una estructura de interfaces, las cuales os mostraré más adelante, que se adapta dinámicamente a la pantalla desde la que se ejecuta, para que no hubiesen cambios imprevistos entre unas y otras.

2.2.2. Acceso a Datos

La aplicación del módulo de Acceso a Datos iba a ser más tedioso y complejo en mi cabeza de lo que acabó siendo, únicamente gracias a la funcionalidad de PlayerPrefs todos los problemas se fueron.

Como ya he explicado sobre esta tecnología, Unity guarda y carga los datos en los archivos del sistema mediante un formato clave-valor de la manera más simple posible, y únicamente con dos comandos puedes acceder y actualizar la información del progreso del usuario.

Así cumpliendo con los requisitos de este módulo.

2.2.3. Programación Multimedia y Dispositivos Móviles

Este es el módulo que más presente está en el proyecto, todos los conocimientos de scripts, programación, lógica de Unity, conceptos de desarrollo de videojuegos...

Todo esto lo he tenido que volcar en este proyecto. Uniendo todos los componentes que hacen de un videojuego algo multiplataforma y multimedia, además de mi búsqueda de hacerlo cómodo y entretenido, no únicamente un museo de buenas prácticas y código limpio y funcional.

2.2.4. Programación de Servicios y Procesos

La funcionalidad de este módulo en el proyecto es para ir al detalle con la elegancia y lo visual, ya que los hilos principales que hacen que el videojuego funcione los ejecuta únicamente Unity.

Aún así Unity también facilita la creación de hilos, usando Coroutines.

El mayor uso que le he dado a estos hilos han sido como temporizador, que el sistema espere 3 segundos de manera paralela para ejecutar una acción.

Por ejemplo:

```
private IEnumerator MostrarGameOver(string escena, float velocidad)
{
    yield return new WaitForSeconds(1.3f);

    SceneManager.LoadScene(escena, LoadSceneMode.Additive);
}
```

```
CambiarVelocidad(velocidad);  
}
```

Aquí la función del hilo es hacer que cuando se ejecute `MostrarGameOver` espere 1.3 segundos antes de ejecutar el código después del `yield return`, lo que hace que quede más estético. Otro uso que le doy es hacer un `yield return` es hacer que espere 3 segundos antes de destruir el objeto de un enemigo, con un código que genera una desaparición muy visual.

3. Desarrollo del proyecto

Ahora vamos a hablar más profundamente de la estructura y la implementación de todo lo que hace que el videojuego funcione correctamente.

3.1. Ambientación

Al comienzo tenía muchas ideas que me ilusionaban para un videojuego, muchos enemigos, consumibles, objetos, equipaciones... Rápidamente me di cuenta de que no era realista, por lo que decidí comenzar por un solo enemigo, el cual casualmente fue el esqueleto que me dio la idea de la ambientación del entorno de mi videojuego... un cementerio.

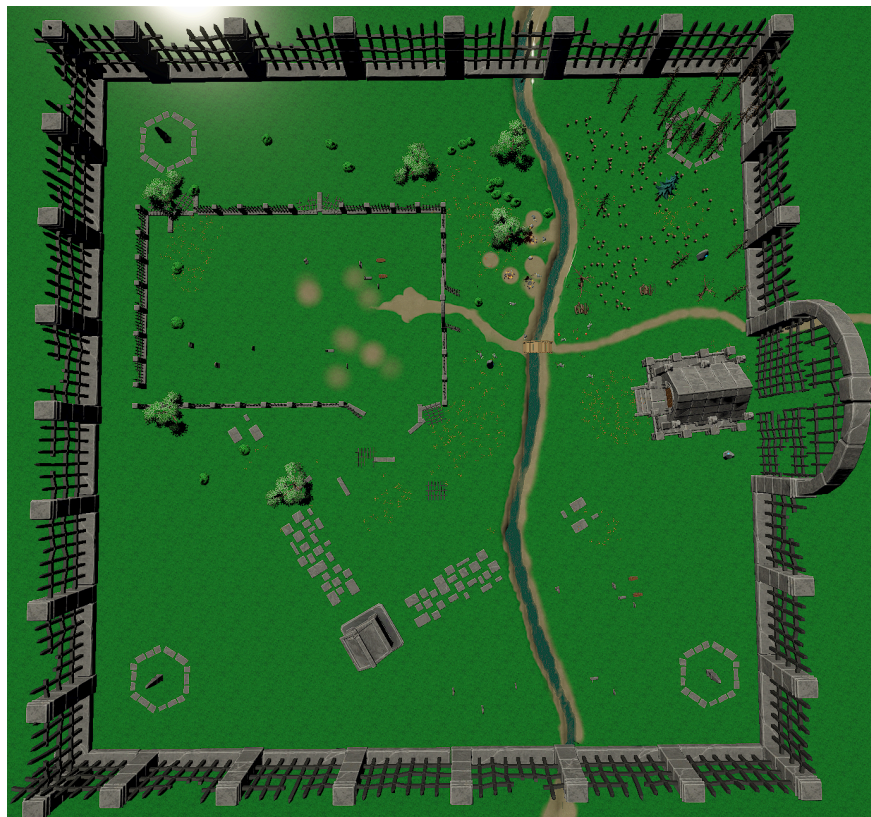


Figura 5: Imagen sobre la ambientación del mapa en la que se desarrolla el videojuego.

Un cementerio, un gran cementerio avallado por grandes verjas, sin escapatoria. Dentro de él podemos ver un par de edificios con la misma temática tenebrosa y antigua, además de unos caminos de piedra y unas verjas más pequeñas destrozadas por "algo". Resaltando todo, hay cuatro estructuras de piedra, demasiado simétricas y estratégicamente colocadas como para ser casualidad ¿verdad?

Más adelante veremos su función.

3.2. Sonido

El sonido también es una muy buena fuente de ambientación, tanto de sonidos naturales, generados por el entorno, como generados por otros objetos.

Antes destacar

Para ambos tipos de sonido he creado un script que hace más fácil gestionar las pistas de audio de cada tipo de sonido, con la información necesaria para reproducirlas correctamente y asignándoles la pista de audio original. Además de añadirlas como una opción en Unity del apartado Create, para crear un objeto de estos scripts.

3.2.1. MusicManager.cs

Este es el encargado de gestionar la música de fondo, que concuerde con la situación en la que estás. Cada menú tiene su música predeterminada, por lo que al cambiar de menú la música cambia.

Al estar presente en todas las escenas del juego decidí aplicar el patrón de diseño Singleton, lo cual es crear una instancia estática, muy llamada Instance en el mundo de C#, para que solo existiese un objeto con este script en todo el proyecto, el cuál será el encargado de gestionar toda la música de fondo de todo el videojuego.

Para reproducir estos sonidos se usa un AudioSource general de toda la escena, no es un sonido 3D.

3.2.2. SoundManager.cs

Este es el script que se encarga de gestionar efectos de sonido, una única vez por ejecución. Por la forma en el que lo he diseñado cada escena que tendrá la posibilidad de generar este tipo de sonidos tiene este script asignado a un objeto, además de un AudioSource que tendrá cada objeto con la capacidad de ejecutar estos sonidos.

He modificado los valores del Spatial Blend en todos los AudioSource de objetos externos al personaje de la escena de Juego, lo cual hace que tenga un cierto porcentaje de sonido 3D, para que tenga cierto punto de realismo e inmersión. Manteniendo los sonidos de menú completamente 2D.

3.3. Escenas

Para el flujo del videojuego he decidido crear cuatro escenas, Menu, Mercado, Juego y GameOver con las que el usuario puede moverse y tener la experiencia total del videojuego.

3.3.1. Flujo entre escenas

El flujo entre escenas se puede ver claramente en la siguiente imagen:

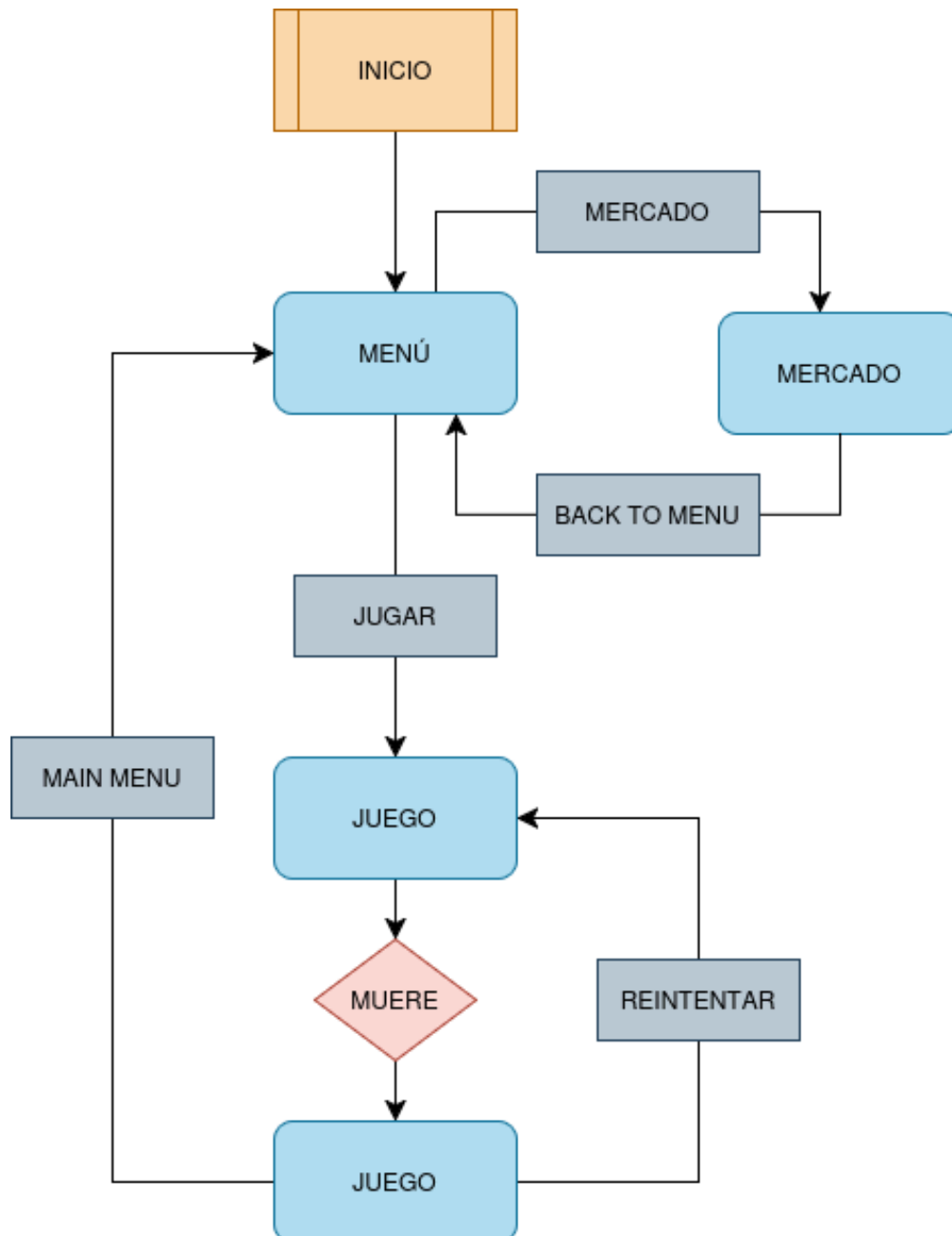


Figura 6: Imágen sobre el flujo de escenas del proyecto.

Como puedes ver, todas las escenas se mueven entre sí por medio de un botón que se acciona... Todas, menos una.

SceneAdditive Este término es el que se utiliza para indicar que quieres añadir una escena como aditiva, osea que se sobreponga a la actual.

Y este es el modo que he usado para cargar la escena de GameOver.



Figura 7: Muestra de la sobreposición de la escena GameOver sobre la escena Juego.

Utilizando una coroutine como las que he mencionado antes para que tarde 3 segundos en aparecer, además de aparecer con un FadeOn, quedando muy estético.

Algo que aprendí mientras implementaba esta funcionalidad es que al cargar una escena sobre otra el EventSystem que usa es el de la escena que había antes, por lo que no hace falta añadirla, entre otros elementos y funciones que actúan parecido.

3.4. MenuSystem.cs

Antes de continuar, creo que es importante explicar la funcionalidad del script MenuSystem, ya que ayuda a que la navegación entre escenas funcione fácilmente.

Este script tiene un método para cargar cada una de las escenas, además de uno para cerrar el videojuego, que se activa con el botón "Salir" del menú.

Cada método tiene 3 partes:

1. **Activación o desactivación del ratón:** Esta parte desactiva el ratón al comienzo del método que activa la escena "Juego" y lo activa en el resto de escenas, ya que es necesario.
2. **StartCoroutine:** Ejecuta una coroutine para cambiar a la escena especificada con retraso, para un efecto de carga.
3. **MusicManager:** Finalmente se ejecuta la música de fondo de esa escena.

Cada uno de estos métodos se activan especificando la acción a ejecutar en el apartado de "On Click" de los botones.

3.5. Escena “Menu”

Esta es la escena más simple de todas.

3.5.1. Diseño

- Con un diseño de un fondo con un Aspect Ratio Filter que hace que no se achate la imagen, sino que amplíe sobre la zona de la imagen más significativa cuando la resolución de la pantalla lo requiera.
- El título del nombre del videojuego es un elemento aparte para no afectar a su resolución y que siempre sea visible con calidad. Siempre fijo a cierta distancia del borde superior izquierdo.
- Finalmente pasamos a los botones, con un diseño basado en una distancia fija entre ellos y hacia el borde inferior izquierdo de la pantalla.

3.5.2. Implementación

La única implementación de esta escena es el uso del script de MenuSystem para activar el cambio de escena mediante clicar el botón adecuado.

Además del botón “Salir”, que utiliza el mismo script para ejecutar su función.



Figura 8: Imagen de la escena del 'Menu' del proyecto

3.6. Escena “Mercado”

Esta escena es pura interfaz dinámica.

Pero antes, es necesario explicar dos script:

3.6.1. CoinsSystem.cs

- Este script es el que cambia el valor numérico de un panel de monedas. Al cual hay que pasarle el texto que muestra el valor del panel y la cantidad a asignar. Esta cantidad se utiliza únicamente al cargar el objeto en la escena.

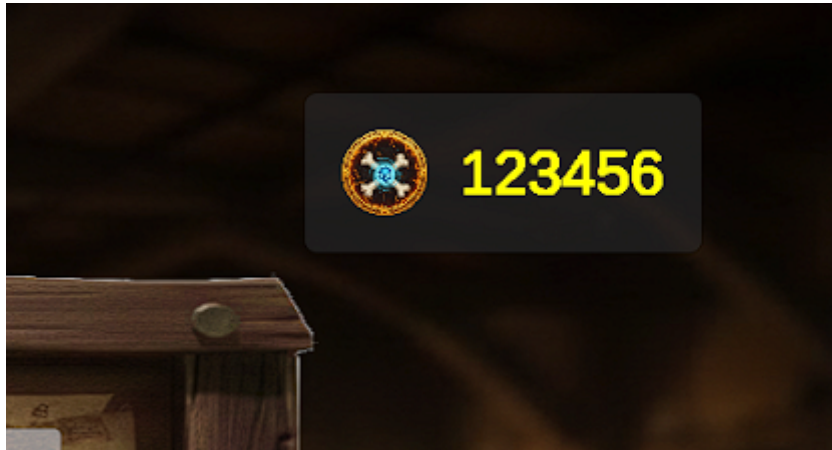


Figura 9: Imagen del panel de CoinsSystem

Todos los objetos que muestren un panel como este llevan un `CoinsSystem.cs` asignado.

- También contiene los métodos `AddCoins` y `SubtractCoins`, que se utilizan para añadir y restar X cantidad de monedas al valor del panel de monedas asignado al script, una vez ya se ha cargado por primera vez el objeto.

3.6.2. UIManager.cs

Este script es muy importante en el apartado de la UI de todo el proyecto. Aunque a su vez es muy sencillo.

Se compone de dos partes fundamentales:

1. **Asignación de los objetos:** A este script se le asignan los objetos a gestionar desde el programa de Unity. Tiene una interfaz embellecida para que se dividan las asignaciones por apartados, así es más intuitivo.
2. **Cambio de estado o valores:** En este segundo apartado ya solo queda modificar cada objeto según requiere, la idea es que todos los demás scripts puedas modificar cualquier apartado de la UI a partir de este script:
 - Si es un panel visual de la escena “Juego” se le asignará el estado que se le pase como parámetro, para que se pueda visualizar en la ejecución del proyecto o no.
 - Si es un texto a modificar se aplicarán los textos que se le pasen como parámetro de la forma predefinida en su método de modificación.

3.6.3. Diseño

- Lo que más destaca de esta escena es el panel de madera que hace de soporte para las Cards de dentro, siendo responsive a cualquier pantalla.

- En la esquina superior derecha tenemos un panel que muestra la cantidad de monedas que tenemos acumuladas, ajustada al tamaño de la pantalla.
- Las tres Cards de mejora de recursos, que tienen:
 - Texto de nivel del recurso.
 - Imagen representativa dinámica al tamaño de la Card y sin achatamientos.
 - Texto de muestra de la mejora de cada estadística del recurso.
 - Panel de muestra del coste de mejora.
 - Botón de mejora del recurso.
- Botón para volver a la escena "Menu" en el centro inferior.

3.6.4. Implementación (MarketSystem.cs)

Sabiendo la distribución de elementos y su agrupación por la escena "Mercado" ya os puedo explicar este script.

Este script se usa únicamente para la mejora de los recursos del usuario, y se dispone en dos grandes bloques:

1. Plasmar datos al panel

Este paso es importante, ya que lo que hace es usar `PlayerPrefs` para buscar y asignar, valores encontrados o por defecto, a todos los textos dinámicos de la escena a través del `UIManager` para que todo sea visual y preciso.

Esto incluye: el nivel del recurso, valores de las estadísticas y coste de mejora.

2. Mejora y actualización

Esta parte tiene un método para la mejora y actualización de cada recurso, cada cual se acciona por el "On Click" de cada botón a su respectivo recurso.

La estructura del funcionamiento de estos métodos es:

1. Asignar el valor de coste a una variable, por medio del `PlayerPrefs`.
2. Ejecutar el método `SubtractCoins` del `CoinsSystem` y almacenar el booleano de éxito en una variable.
3. Reproducir el sonido correspondiente, si se mejoro exitosamente o no, mediante el `SoundManager`.
4. Aplicar los cálculos de incremento de nivel a cada estadística, al valor del nivel mismo y ejecutar un método de `RecalcularCoste()` que modifica y guarda el siguiente coste de mejora de forma exponencial.
5. Almacenar todos estos cambios en la clave correspondiente del `PlayerPrefs`



Figura 10: Imagen de la escena 'Mercado' del proyecto

3.7. Escena “Juego”

Esta es la escena en la que más tiempo pasarás, por ello le he dedicado tiempo a que sea cómoda y no agobien los elementos de la interfaz.

3.7.1. Diseño

En el primer vistazo podremos observar:

- En la parte superior izquierda la barra de salud de nuestro personaje.
- En la parte superior opuesta el panel de monedas que tenemos.
- En el caso de que estemos jugando la versión móvil veremos que en la mayor parte de las esquinas inferiores tendremos botones y un joystick para adecuar la jugabilidad en estos dispositivos.

Conforme vayamos jugando aparecerán otros elementos de la interfaz, como pueden ser:

- El contador de oleadas que aparecerá en la zona superior central.
- Además de las interfaces de interacción con las armas que aparecerán cuando nos acerquemos a ellas.

3.7.2. Implementación

Todos estos elementos están sujetos al script UIManager que afecta a esta escena, tomando el control de cuando aparecen y desaparecen en pantalla, además de cambiar sus valores a tiempo real.

- Comenzando por la barra de vida, tiene su propio script que actualiza la barra verde dependiendo del porcentaje de vida respecto a su vida máxima que le queda.
- En cuanto al panel de monedas, utiliza el script CoinsSystem explicado anteriormente, siendo activada la actualización del valor cuando se detecta la muerte de un enemigo.
- La interfaz móvil se incluye si detecta una pantalla táctil al iniciar la escena.
- Con el texto del contador de oleadas lo maneja otro script llamado WaveManager a través del UIManager, más adelante explicaremos la funcionalidad interna de este.
- Igual ocurre con los textos de interactuar con las armas, pero de manera más compleja. Lo maneja el script Coger_Arma del personaje, y según las entradas de datos que detecte muestra

un texto u otro. Ya que la interfaz de cambio de arma es diferente para móvil, móvil PC o mando. Más adelante se explicará en profundidad.



Figura 11: Imagen de la escena 'Juego' del proyecto

3.7.3. DayNightCycle.cs

Para una mayor ambientación y un poco de sensación tétrica quería implementar un sistema de noche, y para que no fuese estático decidí hacer un ciclo de día y noche, ahí nace este script.

Este script funciona así:

- Tienes dos objetos que generan luz, sol y luna, uno emite luz amarilla/blanca y el otro azul oscura, posicionados opuestamente.
- El script va variando la intensidad de cada objeto conforme se acerca al horizonte, usando parámetros de máximas intensidades y tiempo que tarda en hacer un ciclo, para que sea personalizable. Además de que el mismo genera la rotación de los objetos, de manera sincronizada.



Figura 12: Imagen del estado de noche del proyecto

3.8. Escena “GameOver”

Esta es la escena especial, ya que no tiene implementación, es gestionada en su totalidad por los scripts existentes en la escena “Juego”.

3.8.1. Diseño

El diseño de esta interfaz es simple y estática en si misma, en la ejecución del juego adopta la funcionalidad al igual que cualquier otra escena:

- Una imagen que hace como marco, ocupando toda la pantalla.
- Una imagen bien acomodada que da la noticia del GameOver.
- Y los dos botones para continuar con el flujo de juego tras perder la partida.

Es simple pero hace su función tan bien como las otras.



Figura 13: Imagen de la escena 'GameOver' del proyecto

4. Bugs y soluciones

4.1. SceneAdditive

- El SceneAdditive me ocasionó un problema con el MusicManager, ya que solamente puede haber uno por escena, y al hacer aditiva la escena de GameOver y tener su propio MusicManager además del existente de la escena Juego me detectaba un error, el cual era que existían dos instancias de un objeto que debe ser único.

Por lo que tuve que borrar el MusicManager de la escena GameOver, ya que reutilizaba el de la escena Juego.

- Además la escena GameOver dependía de que la escena Juego se hubiese ejecutado previamente, sino no tendría los elementos funcionales cuando los requería.

Así que solo podía hacer pruebas basándome en el flujo natural de juego, no solo en esa escena.

Índice de Figuras e Imágenes

Figura 1	Imagen visual de los elementos del NavMesh	3
Figura 2	Imagen real del flujo entre botones generados por UI Navigation del proyecto	4
Figura 3	Imagen explicativa del uso de InputSystem	4
Figura 4	Imagen sobre otro ejemplo de uso de Blend Tree 1D	5
Figura 5	Imagen sobre la ambientación del mapa en la que se desarrolla el videojuego.	8
Figura 6	Imágen sobre el flujo de escenas del proyecto.	10
Figura 7	Muestra de la sobreposición de la escena GameOver sobre la escena Juego.	11
Figura 8	Imagen de la escena del 'Menu' del proyecto	12
Figura 9	Imagen del panel de CoinsSystem	13
Figura 10	Imagen de la escena 'Mercado' del proyecto	15
Figura 11	Imagen de la escena 'Juego' del proyecto	16
Figura 12	Imagen del estado de noche del proyecto	16
Figura 13	Imagen de la escena 'GameOver' del proyecto	17

Índice de Tablas
